AWS Cognito Identity Pool Issues & Incorrect Solution Methods

Pre-requisites:

- Windows 10 or later or Windows Server 2016 or later
- An Intel Pentium 4 processor or later that is SSE3 capable

Starting Point:

At the beginning of 8/10/2025 there were various bugs that needed immediate fixing in order to perform further work on the authentication functionality of the backend. These bugs can be summed up with the console output from the browser running the web app as seen below.

Method 1: New AuthRedirect.tsx file & Layout.tsx Code Changes

This method sought out to fix the following:

GET https://lab.crusaders.clockwork.vip/oauth2/idpresponse/?code=3ff8b456-689c-468d-a0fa-a69ec4b4ce6d&state=dJeN6zLehnEmmb2mHflOYnG4q0M6hvdB 404 (Not Found)

Home.tsx:12 Backend URL: https://api.lab.crusaders.clockwork.vip

favicon.ico:1

GET https://lab.crusaders.clockwork.vip/oauth2/idpresponse/favicon.ico 404 (Not Found)

useFetchUser.ts:39

POST https://cognito-identity.ap-southeast-2.amazonaws.com/ 400 (Bad Request)

useFetchUser.ts:35 Error fetching user info: NotAuthorizedException: Unauthenticated access is not supported for this identity pool.

at IdentityIdProvider.ts:58:15

at async Pk (IdentityIdProvider.ts:28:11)

at async Ek.getCredentialsAndIdentityId (credentialsProvider.ts:45:15)

at async It.fetchAuthSession (index.ts:44:13)

at async xs (fetchUserAttributes.ts:14:13)

at async useFetchUser.ts:21:15

It did so initially by providing a route for /oauth2/idpresponse, as react was receiving that route but there was no code anywhere to handle that routing. This was completed by creating a new file src/pages/AuthRedirect.tsx that simply waited for the user to be logged in and redirected the user to the route accordingly. Furthermore, to support this change the Layout.tsx file had an extra route added. This resulted in a error however, displayed below:

2025-10-13T11:40:06.728Z [INFO]: Attempted import error: 'Auth' is not exported from 'aws-amplify' (imported as 'Auth').

To fix this, the AuthRedirect.tsx file was updated according to the changes below:

import { Auth } from "aws-amplify";

To:

import { handleSignInRedirect } from "aws-amplify/auth";

But this change only resulted in a similar error, shown below:

2025-10-13T11:52:50.043Z [INFO]: Attempted import error: 'handleSignInRedirect' is not exported from 'aws-amplify/auth' (imported as 'handleSignInRedirect').

To resolve this error, the “useEffect” function was changed, and the import was switched back according to the following:

import { handleSignInRedirect } from "aws-amplify/auth";

To:

import { Auth } from "aws-amplify";

And:

useEffect(() => {

  const completeLogin = async () => {

    try {

```
      // Complete Cognito Hosted UI redirect flow
      await handleSignInRedirect();
      console.log("✅ Successfully signed in via redirect");
      navigate("/"); // redirect to homepage
    } catch (error) {
      console.error("❌ Error completing OAuth redirect:", error);
    }
  };
```

To:

```
  useEffect(() => {
    const completeLogin = async () => {
      try {
        // This tells Amplify to complete the Cognito redirect flow
        const user = await Auth.currentAuthenticatedUser();
        console.log("✅ Already signed in:", user);
        navigate("/");
      } catch {
        try {
          // If not signed in yet, finish the hosted UI auth flow manually
          const result = await Auth.federatedSignIn();
          console.log("✅ Successfully signed in via redirect:", result);
          navigate("/");
        } catch (error) {
```

```
      console.error("❌ Error completing OAuth redirect:", error);
    }
  }
};
```

Method 2: aws-exports.js, amplifyconfiguration.json, App.js & index.js Code Changes

This method involved altering the code in certain configuration files and files concerned with the initial loading of the website. This was to ensure from the beginning of the building of the application that it has the correct identity pool ID so it uses it instead of any other ones. This solution attempted to fix the following errors:

content-script.js:22 Document already loaded, running initialization immediately

content-script.js:4 Attempting to initialize AdUnit

content-script.js:6 AdUnit initialized successfully

/oauth2/idpresponse/favicon.ico:1

Failed to load resource: the server responded with a status of 404 ()

cognito-identity.ap-...-2.amazonaws.com/:1

Failed to load resource: the server responded with a status of 400 ()

useFetchUser.ts:35 Error fetching user info: ResourceNotFoundException: IdentityPool 'ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b' not found.

at async useFetchUser.ts:21:15

Firstly, the aws-exports.js file was updated according to the changes below:

"aws_cognito_identity_pool_id": "ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b",

To:

"aws_cognito_identity_pool_id": "ap-southeast-2:6d5ea010-9747-46ca-a38a-afa2d0aa7403",

This was to ensure that this file was not pointing to the wrong identity pool causing the web application to use the wrong one and throw errors. Following this change, more instances of the wrong identity pool ID being used were searched for using the “Find in Files” function apart of VSCode, resulting in another problematic usage being found out. This was in the amplifyconfiguration.json file, and it was consequently updated with the following:

"aws_cognito_identity_pool_id": "ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b",

To:

"aws_cognito_identity_pool_id": "ap-southeast-2:6d5ea010-9747-46ca-a38a-afa2d0aa7403",

Following this, considering the error remained occurring it was very confusing to know where the error was originating. As a result, debugging statements were added to useFetchUser.ts according to the changes below:

```
import { signOut, fetchUserAttributes } from "aws-amplify/auth";

import { useEffect, useState } from "react";



interface UserAttributes {

  email: string;

  identities?: string;

  picture?: string;

}



export function useFetchUser(): [

  user: UserAttributes | null,

  handleSignOut: () => void,

] {

  const [user, setUser] = useState<UserAttributes | null>(null);



  // Fetch user attributes on component mount
```

```
useEffect(() => {
  const fetchUser = async () => {
    try {
      // Fetch the user attributes (expecting an object with key-value pairs)
      const attributes = await fetchUserAttributes(); // Log this structure to verify


      // Assuming the response is an object with `email` and `identities` fields.
      const email = attributes.email ?? "Unknown";
      const identities = attributes.identities ?? "Unknown";
      const picture = attributes.picture ?? undefined;


      // Set the mapped user attributes in the state
      setUser({
        email,
        identities,
        picture,
      });
    } catch (error) {
      console.error("Error fetching user info:", error);
    }
  };


  fetchUser();
}, []);
```

```
  const handleSignOut = async () => {
    try {
      await signOut();
    } catch (error) {
      console.error("Error signing out:", error);
    }
  };

  return [user, handleSignOut];
}
```

To:

```
import { signOut, fetchUserAttributes, fetchAuthSession, getCurrentUser } from "aws-amplify/auth";
import { Amplify } from "aws-amplify";
import { useEffect, useState } from "react";

interface UserAttributes {
  email: string;
  identities?: string;
  picture?: string;
}

export function useFetchUser(): [
```

```
  user: UserAttributes | null,
  handleSignOut: () => void,
] {
  const [user, setUser] = useState<UserAttributes | null>(null);

  useEffect(() => {
    const fetchUser = async () => {
      try {
        console.group("🧭 useFetchUser Debug Log");

        // 1️⃣ Print Amplify config at runtime
        const amplifyConfig = Amplify.getConfig();
        console.log("📦 Amplify Config:", amplifyConfig);

        // 2️⃣ Print current auth session details
        const session = await fetchAuthSession();
        console.log("🔑 Auth Session:", session);

        // 3️⃣ Attempt to get current user info
        try {
          const userInfo = await getCurrentUser();
          console.log("👤 Current user:", userInfo);
        } catch (innerError) {
```

```
    console.warn("⚠️ getCurrentUser failed:", innerError);
  }

  // 4️⃣ Try to fetch user attributes
  console.log("📤 Fetching user attributes...");
  const attributes = await fetchUserAttributes();
  console.log("✅ Raw user attributes:", attributes);

  const email = attributes.email ?? "Unknown";
  const identities = attributes.identities ?? "Unknown";
  const picture = attributes.picture ?? undefined;

  setUser({ email, identities, picture });

  console.groupEnd();
} catch (error: any) {
  console.groupEnd();
  console.error("❌ Error fetching user info:", error);

  // 5️⃣ Print additional clues if the error is a Cognito one
  if (error.name === "ResourceNotFoundException" || error.message?.includes("IdentityPool")) {
    console.error("🚨 Possible cause: Incorrect Identity Pool ID in configuration");
  }
```

```
      if (error.$metadata) {
        console.error("🔍 AWS Metadata:", error.$metadata);
      }
    }
  };

  fetchUser();
}, []);

const handleSignOut = async () => {
  try {
    console.log("🚪 Signing out user...");
    await signOut();
    console.log("✅ Signed out successfully");
  } catch (error) {
    console.error("Error signing out:", error);
  }
};

return [user, handleSignOut];
}
```

This did not resolve any of the issues.

Method 3: Uninstalling & Reinstalling Node.js

This method was used to attempt to fix the following errors, namely involving one stating that the web application is using an old identity pool ID, despite the ID being found nowhere in the files in the repository:

POST https://cognito-identity.ap-southeast-2.amazonaws.com/ 400 (Bad Request)
useFetchUser.ts:35

Error fetching user info: ResourceNotFoundException: IdentityPool 'ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b' not found. at async useFetchUser.ts:21:15

The reasoning behind this method is that if a web application is deployed using AWS Amplify, then Amplify can sometimes use the old built bundle in the backend server itself.

To resolve this, the following commands are entered into the console:

cd /apps/ClockWorkWeb/lab/frontend

rm -rf node_modules/.cache

npm install

npm run build

Doing this should clear the cache which can store the old Amplify or Webpack configurations, including environment settings such as the identity pool ID, and redeploy the application. However, attempting to run these commands resulted in a subsequent need to reinstall Node.js, leading to a lot of problems regarding that. This was seen with the server initially not having enough memory to reinstall Node.js, and as a result the following commands were entered into the console to resolve that:

rm -rf node_modules

rm -rf package-lock.json

rm -rf node_modules/.cache

npm cache clean –force

export NODE_OPTIONS=--max-old-space-size=4096

npm install

These commands failed with the console outputting “Killed” after hanging for a long time. To resolve this, it was thought that the Node.js version may be too old, considering the reinstallation produced warnings that stated:

current: { node: 'v16.20.2', npm: '8.19.4' }

required: { node: '^18.14.0 || ^20.0.0 || ^22.0.0 || >=24.0.0' }

Consequently, the following commands were entered into the console:

nvm install 20

nvm use 20

nvm alias default 20

This only lead to further issues, as it was only realized after this point that Amazon Linux 2, which the server runs on has glibc 2.26, while Node 18+ and Node 20 require a later version of glibc, with 2.27-2.28 working with it. This is why the “node -v” command failed after the installation of Node 20, with the console outputting:

node: /lib64/libm.so.6: version GLIBC_2.27' not found (required by node)

node: /lib64/libc.so.6: version GLIBC_2.28' not found (required by node)

The options to resolve this issue included either:

- Moving the entire project to Amazon Linux 2023
- Moving the build inside a Docker container with Node 20

As a result, this solution became much less attractive, with the following commands being entered to revert the changes made from this method:

nvm use 16

nvm alias default 16

export NODE_OPTIONS="--max-old-space-size=4096"

rm -rf node_modules package-lock.json

npm install

npm run build

scp -r build/* ec2-user@13.211.122.157:/var/www/html/

sudo chown -R apache:apache /var/www/html

sudo chmod -R 755 /var/www/html

sudo systemctl restart httpd

This method did not result in any progress being achieved.

Method 4: Changing Amplify Rules, Hard Refreshing & Disabling the Browser's Cache – Showed Some Progress

This method was used to attempt to fix the following errors, namely involving one stating that the web application is using an old identity pool ID, despite the ID being found nowhere in the files in the repository:

POST https://cognito-identity.ap-southeast-2.amazonaws.com/ 400 (Bad Request)
useFetchUser.ts:35

Error fetching user info: ResourceNotFoundException: IdentityPool 'ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b' not found. at async useFetchUser.ts:21:15

By the end of this method, the errors had changed from these previous ones to the following:

useFetchUser.ts:27 POST https://cognito-identity.ap-southeast-2.amazonaws.com/ 400 (Bad Request)

useFetchUser.ts:52 ❌ Error fetching user info: NotAuthorizedException: Unauthenticated access is not supported for this identity pool. at async useFetchUser.ts:27:15

It is unclear what part of this method caused it to change in this way.

This method began by simply checking if disabling the cache in the browser and hard refreshing would force the browser into using the updated identity pool ID. Then, Amplify's "Rewrites and redirects" were changed. This first took place with the following modification:

From:

```
[
 {
```

```
  "source": "https://crusaders.clockwork.vip ",
  "status": "302",
  "target": "https://www.crusaders.clockwork.vip "
 },
 {
  "source": "/<*>",
  "status": "404-200",
  "target": "/index.html"
 }
]
```


To:


```
[
 {
  "source": "https://crusaders.clockwork.vip ",
  "status": "302",
  "target": "https://www.crusaders.clockwork.vip "
 },
 {
  "source": "/<*>",
  "status": "200",
  "target": "/index.html"
 }
]
```

This change hinges on that the new rule forces every URL to take up a fresh index.html, causing the new build containing the updated identity pool ID to be used. This alteration caused an issue however, with the web application not displaying entirely, instead showing a completely blank white page, with the console outputting:

main.a84457c7.js:1 Uncaught SyntaxError: Unexpected token '<'Understand this error

content-script.js:22 Document already loaded, running initialization immediately

content-script.js:4 Attempting to initialize AdUnit

content-script.js:6 AdUnit initialized successfully

manifest.json:1 Manifest: Line: 1, column: 1, Syntax error.

To resolve this, further rules were added resulting in the following for “Rewrites and redirects”:

```
[
 {
   "source": "/manifest.json",
   "target": "/manifest.json",
   "status": "200"
 },
 {
   "source": "/static/<<*>>",
   "target": "/static/<<*>>",
   "status": "200"
 },
 {
   "source": "https://crusaders.clockwork.vip",
   "target": "https://www.crusaders.clockwork.vip",
   "status": "302"
```

```
 },
 {
   "source": "/<*>",
   "target": "/index.html",
   "status": "200"
 }
]
```

However, this change did not work either, returning the same blank white page and the same errors. This change may not have done anything because the build was not redeployed on Amplify. Seeing the web page not come back after this second change, I got worried, reverted the changes to the “Rewrites and redirects” and redeployed the build on Amplify. This resulted in the web application coming back, and the console returning the following as a final output for the entire identity pool ordeal:

useFetchUser.ts:27 POST https://cognito-identity.ap-southeast-2.amazonaws.com/ 400 (Bad Request)

useFetchUser.ts:52 ❌ Error fetching user info: NotAuthorizedException: Unauthenticated access is not supported for this identity pool. at async useFetchUser.ts:27:15